



ASSIGNMENT REPORT

RESTFUL BLOG API

Semester:	Programming Fundamentals
Class:	WD1113
Instructor:	Cuong Nguyen Dinh
Members:	Phan Nhat Quan (Alexander Phan)

MỤC LỤC

1. Giới thiệu dự án
 2. Mục tiêu và phạm vi bài làm
 3. Công nghệ sử dụng
 4. Thiết kế kiến trúc hệ thống
 5. Thiết kế dữ liệu
 6. Triển khai Backend (RESTful API)
 7. Triển khai Frontend (nếu có)
 8. Các chức năng chính đã thực hiện
 9. Cách kiểm tra và kiểm thử hệ thống
 10. Thách thức gặp phải và hướng giải quyết
 11. Kết quả đạt được
 12. Kết luận và hướng phát triển
 13. Phụ lục (screenshot, Postman, tài liệu tham khảo)
-

1. Thông tin chung

- Môn học: Lập trình Web với Node.js
 - Đề tài: Xây dựng RESTful Blog API với Authentication & Authorization
 - Sinh viên thực hiện: Phan Nhật Quân
 - MSSV: 1140101240014
 - Lớp: WD1113
 - Ngày nộp:
 - Repository: <https://github.com/AlexanderPhan04/NodeJS-Programming>
-

2. Mục tiêu bài làm

Mục tiêu của bài tập là xây dựng một hệ thống API cho blog cá nhân cho phép:

- Đăng ký và đăng nhập người dùng
- Tạo, sửa, xóa bài viết
- Phân quyền owner/admin khi sửa/xóa
- Upload ảnh cho bài viết
- Tìm kiếm, lọc, phân trang danh sách bài viết

Bên cạnh đó, hệ thống phải đảm bảo bảo mật cơ bản bằng JWT, hash mật khẩu và xử lý lỗi thống nhất.

3. Công nghệ sử dụng

- Node.js
 - Express
 - MongoDB
 - Mongoose
 - jsonwebtoken (JWT)
 - bcryptjs (hash password)
 - multer (upload file ảnh)
 - Joi (validate dữ liệu đầu vào)
 - express-rate-limit (giới hạn tần suất request)
 - morgan (HTTP logging)
-

4. Thiết kế kiến trúc

Dự án được tổ chức rõ theo hướng tách lớp:

- routes/: khai báo endpoint
- controllers/: điều phối request/response
- services/: xử lý nghiệp vụ
- repositories/: truy cập dữ liệu
- models/: định nghĩa schema Mongoose
- middlewares/: auth, upload, validate, error handling

Luồng xử lý chính:

1. Request vào route
 2. Qua middleware (JWT, upload, validate)
 3. Controller gọi service
 4. Service áp nghiệp vụ + gọi repository
 5. Repository thao tác MongoDB qua model
 6. Trả response JSON thống nhất
-

5. Thiết kế dữ liệu

5.1. User

- username (unique)
- email (unique)
- passwordHash
- role (user hoặc admin)
- createdAt, updatedAt

5.2. Post

- title
 - content
 - author (ObjectId tham chiếu User)
 - imageUrl
 - tags (mảng string)
 - createdAt, updatedAt
-

6. Danh sách endpoint đã triển khai

6.1. Authentication

Method	Endpoint	Mô tả
POST	/api/auth/register	Đăng ký tài khoản mới
POST	/api/auth/login	Đăng nhập, trả về JWT

6.2. Post

Method	Endpoint	Mô tả
GET	/api/posts	Lấy danh sách bài viết (phân trang)
GET	/api/posts/:id	Lấy chi tiết bài viết
POST	/api/posts	Tạo bài viết (JWT, có thể upload ảnh)
PUT	/api/posts/:id	Cập nhật bài viết (JWT, owner/admin)
DELETE	/api/posts/:id	Xóa bài viết (JWT, owner/admin)
GET	/api/posts/my-posts	Lấy bài viết của user hiện tại (JWT)
GET	/api/posts/search?q=...&tag=...	Tìm kiếm theo từ khóa và tag

Endpoint bổ sung:

- GET /api/health để kiểm tra trạng thái service
-

7. Nghiệp vụ bắt buộc đã xử lý

1. Authentication bằng JWT:

- Đăng nhập trả token
 - Middleware kiểm tra Authorization: Bearer <token>
 - 2. Authorization:
 - Chỉ chủ bài viết hoặc admin được sửa/xóa
 - 3. Password security:
 - Mật khẩu hash bằng bcryptjs trước khi lưu
 - 4. Upload ảnh:
 - Dùng multer, lưu ảnh vào public/uploads
 - 5. Validation:
 - Dùng Joi cho auth payload, post payload, query params
 - 6. Pagination:
 - Hỗ trợ ?page=1&limit=10
 - 7. Search/filter:
 - Hỗ trợ ?q=keyword và ?tag=Node.js
 - 8. Error handling:
 - Middleware xử lý lỗi toàn cục, trả status code + JSON phù hợp
-

8. Cấu hình và vận hành

- Port mặc định: 4000
- MongoDB URI mặc định: mongodb://127.0.0.1:27017/NodeJS-Programming

Có seed admin mặc định nếu hệ thống chưa có admin:

- Email: admin@example.com
 - Password: Admin@123456
-

9. Kết quả kiểm thử

Đã kiểm thử thủ công bằng Postman theo collection đi kèm:

- postman/blog-api.postman_collection.json

Các luồng đã test:

- Register, login lấy JWT
- Tạo bài viết có upload ảnh
- Lấy danh sách bài viết có phân trang
- Search theo q và tag
- Lấy danh sách my-posts
- Cập nhật/xóa bài theo quyền owner và admin
- Kiểm thử lỗi khi thiếu token, token sai, hoặc vi phạm quyền

Kết quả: hệ thống phản hồi đúng dữ liệu và status code theo yêu cầu.

10. Khó khăn và cách giải quyết

Khó khăn đầu tiên là quản lý phân quyền owner/admin cho thao tác cập nhật và xóa bài viết.

→ Giải pháp: tách rõ hàm kiểm tra quyền trong service và tái sử dụng cho PUT/DELETE.

Khó khăn thứ hai là xử lý dữ liệu tags khi gửi dạng multipart/form-data (CSV hoặc JSON string).

→ Giải pháp: chuẩn hóa dữ liệu ở middleware trước khi validate.

Khó khăn thứ ba là đảm bảo đồng nhất format lỗi.

→ Giải pháp: sử dụng middleware error handler chung.

11. Đối chiếu với tiêu chí đề bài

- Authentication & JWT: Đạt
- Authorization owner/admin: Đạt
- CRUD Post + upload ảnh: Đạt
- Validation input + error handling: Đạt
- Tìm kiếm + phân trang: Đạt
- Cấu trúc code rõ ràng: Đạt
- README + Postman collection: Đạt

Điểm cộng:

- Dùng Mongoose + schema
 - Có rate limiting
 - Có cấu trúc tách lớp rõ ràng
-

12. Kết luận

Bài làm đáp ứng đầy đủ các yêu cầu chính của đề bài về chức năng, bảo mật cơ bản và tổ chức mã nguồn.

API hoạt động ổn định với MongoDB, có thể mở rộng thêm comment, test tự động và deploy cloud trong tương lai.

13. Phụ lục screenshot

- Screenshot 1: Register/Login trả JWT

- Screenshot 2: Tạo bài viết có upload ảnh
- Screenshot 3: Search + pagination
- Screenshot 4: Cập nhật/xóa bài theo quyền owner/admin
- Screenshot 5: Lỗi khi token không hợp lệ hoặc không đủ quyền